

Team Name	SEA	Assessment Item	11C
-----------	-----	-----------------	-----

The use of the following is permitted on this course only with full disclosure of how it was used and with proper attribution, you must make clear what is your teams and/or your own work:

- Open-source code and libraries (excluding previous DECO projects).
- Open data sets.
- Royalty free, free to use icons, images.
- Use of generative AI code generation tools such as Copilot and others.
- Use of generative AI tools such as ChatGPT to assist with essay writing (but only with a complete list of the prompts used).
- Use of AI grammar checkers.

We declare that:

1. All team members confirm and acknowledge they have signed off on the submission of this work. Each team member agrees that:
 - The work submitted accurately represents the work of the team.
 - Changes to the final submission have not been made without approval from the entire team.
2. This work, except where otherwise indicated and fully attributed, is our own original work.
3. This work has not been submitted for any degree or examination at any other university and/or for any other course at UQ (*except where explicitly permitted*).
4. Third party material – code/data/tutorials/work
 - This work **does not** contain any other persons' work, open-source work, and/or generative artificial intelligence code, data, images, graphs, tables, or any other information.
 - This work **does** contain work inspired by, drawn from, copied from, derived from (or similar) other persons' work, open-source work, and/or generative artificial intelligence (Gen-AI) code, data, images, graphs, tables, or any other information. Such instances have been indicated below AND are appropriately disclosed and attributed in place

where the work occurs (in-text citations, reference list, code comments/braces, etc). For Gen-AI, such as ChatGPT, acknowledge below and include the full set of prompts and responses within the appendix.

5. Third party material - writing

- This work **does not** contain other persons’ and/or Gen-AI writing.

☒

○ This work **does** contain other persons’ writing and their words have been re-written, but the general information has been attributed and referenced to them as an in-text citation and in the reference list.

☐

○ This work **does** contain other persons’ writing and their exact words have been used within quotation marks and has been attributed and referenced to them as an in-text citation and in the reference list.

☐

○ This work **does** contain Gen-AI work (with or without modification). Gen-AI has been used to aid writing. The sections it has been used in are listed below. Include the full set of prompts and responses within the appendix.

☒

†

○ This work **has not** been refined, revised, translated, transformed, or similar by any AI tool.

☐

○ This work **has** been refined, revised, translated, transformed, or similar by any AI tool. The tools and description of use are listed below.

☒

†

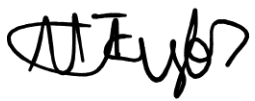
6. We have checked this work to ensure that there are no instances of academic misconduct, including plagiarism herein.

☒

The University defines Academic Misconduct as involving ‘a range of unethical behaviours that are designed to give a student an unfair and unearned advantage over their peers’. The University takes Academic Misconduct very seriously and any suspected cases will be investigated through the University’s standard [policy](#).

We hereby declare that we understand the meaning of academic misconduct and recognise that the submission of work for assessment that includes instances of academic misconduct is an offence that may result in disciplinary action being taken against us.

Name	Signature	Date
Su Yi Tong		14/10/2025

Hui Ying Chia		14/10/2025
Rae Lynn Tan	Raelynn	14/10/2025
Fayza Aqila Bachtiar		14/10/2025
Ralfazza Rajariandhana		14/10/2025
Max Taylor		14/10/2025

SEA-together-at-a-distance

App Flow Diagram

Below is an basic app flow diagram for our project so far.

- User shall entercode to enter a certain meeting and will obliged to fill in a survey
- Survey; this is to obtain their interest. The survey will be static questions (for now) with fields of name, interest (where they can fill in anything, a hobby, a pet, etc) and expertise (a drop down, filled expertise field quite vast within the database)
- After submitting the survey, user will be directed to lobby, where they can see all participants name and interest
- Bingo game will only appear at ice breaking session, enabled by host

TODO & Future Suggestions- Tasks and Future Plans

Tasks

1. Finish backend API for all models and all routes - this is discussed in the backend docuementation.
2. Connect API to more components in frontend and test functionality.
3. Websockets updates and fixes - Updates when player joins or exits.
4. Update UI in zoom up to be more user friendly.
5. Provide tests and check functionality.

Future Enhancements

1. *Real-Time Matching* - Integrate backend interest-matching algorithm
2. *Score Persistence* - Save game results to database
3. *Session State* - Track active sessions and reconnection
4. *Analytics* - Track user engagement and connection outcomes
5. *Advanced Filters* - Allow filtering participants by expertise/interest
6. *Replay/Feedback* - Show completed interactions and suggest new connections

Known Limitations

- Bingo game participants are currently hardcoded (5 dummy players)
- (TODO) Integrate backend matching algorithm
- (TODO) Store completed interactions and scores in database
- (TODO) Add real-time updates for multi-user sessions

Frontend Documentation (React Web)

Overview

The HereToo frontend is a React-based web application designed to facilitate meaningful connections between online and offline participants during hybrid meetings. It uses React Router for navigation and integrates with the Zoom SDK for app-based functionality.

Figma Mockup

A figma Mockup was created to prototype the frontend. The link to view the figma can be found here:

<https://www.figma.com/design/BhTdkRkXE4tteLlgGexSii/deco3801?node-id=0-1&t=uPv41ctr1fWSpkfu-1>

User Flows

Host Flow

/ (Home) → /entercode (host code) → /schedule → /host → Zoom App Integration

What happens:

1. **Home (/)** - Host enters application and views home page
2. **Enter Code (/entercode)** - Host enters their host/admin code to authenticate
3. **Schedule (/schedule)** - Host views and manages scheduled meetings and sessions
4. **Host (/host)** - Host accesses control panel to manage games and participants
5. **Zoom Integration** - Host launches interactive sessions with integrated Zoom SDK

Participant Flow

/ (Home) → /entercode (user code) → /survey → /eventmap → /bingo

What happens:

1. **Home (/)** - Participant starts at landing page
 2. **Enter Code (/entercode)** - Participant enters their user/meeting code to join a session
 3. **Survey (/survey)** - System collects participant interests and expertise to facilitate matching
 4. **Avatar Creation (/avatar)** - Participant creates their own personalised avatar to represent them on the event map
 5. **Event Map (/eventmap)** - Participant explores interactive map of other participants with similar interests
 6. **Bingo Game (/bingo)** - Participant plays matching game to connect with others and break the ice
-

Bingo

Below is a rough sketch of the bingo flow:

- Bingo will be filled with 5 names of people user to interact, later on will be appointed to zoom (if one of them online or both online) or direct meet if offline
 - Conversation question will prompt from the interaction screen and both user will need to fill in the answer to end the interact session and obtain score
 - Finish interacting with all 5 will BINGO!
 - Score will be live updated in the scoreboard for motivation purposes, first one to BINGO! Will be on top of the scoreboard
 - Bingo was designed to connect users based on their interests.
-

Route Structure

Routes WITHOUT Header/Footer

These are full-screen or standalone pages:

- **/** - Home landing page
- **/entercode** - Join meeting with code entry
- **/test** - Development/testing page

Routes WITH Header/Footer

These routes use the `MainLayout` wrapper which includes the HereToo header and footer:

- `/schedule` - Host schedule management
- `/mappage` - Map/lobby page
- `/avatar` - Avatar creation tool
- `/eventmap` - Interactive event map (fullscreen layout)
- `/survey` - Interest and expertise survey
- `/bingo` - Bingo matching game
- `/host` - Host control panel

Core Features & Components

Feature	Route	Purpose	What It Does	User Experience
Interest Survey	<code>/survey</code>	Gathers participant information to power the matching algorithm	• Collects user interests through form submission • Records expertise area selections • Sends data to backend via <code>POST /players/{player_id}/survey</code> • Interests normalized and stored in database	• User fills out form with interests (e.g., "AI/Machine Learning", "Web Development") • Selects primary area of expertise • Form validation ensures required fields completed • After submission, routed to event map
Avatar Creation	<code>/avatar</code>	Allows users to create a personalized 3D avatar that represents them on the event map	• Integrates Ready Player Me platform for avatar customization • Enables users to adjust clothing, facial features, and body attributes • Avatar is later rendered on the Event Map to represent the participant visually	• User customizes their avatar appearance interactively • Can preview changes in real-time (e.g., rotating model) • Saves and confirms design before proceeding • Seamless transition to Event Map after creation
Event Map	<code>/eventmap</code>	Visual exploration and awareness of other participants	• Displays participants as avatars on interactive map • Shows distance/proximity to other users • Displays participant profiles on hover/click • Allows filtering by interest areas	• User sees their avatar and others in spatial layout • Can click on other participants to view profiles • Can see commonalities in interests • Prepares user for Bingo game by introducing available connections

Feature	Route	Purpose	What It Does	User Experience
Bingo Game	<code>/bingo</code>	Structured icebreaker game to facilitate real conversations between matched participants	• Displays 5 matched participants as cards/avatars in grid • User clicks participant to view profile • Opens question modal for conversation starter selection • AI suggestions provide pre-written prompt options • Tracks answers from both participants • Awards 10 points per completed interaction • Win condition: 5 completed interactions	• Player Selection - Click unmatched participant card • Profile View - Modal shows interest, expertise, attendance mode • Question Selection - Choose or enter conversation question • Breakout Room - Two-card interface for question/answer exchange • Score Tracking - Progress bar updates with interactions • Win Condition - Final score displayed after 5 interactions

Component Architecture

App.jsx (Main router)

```

├─ MainLayout (Header + Footer wrapper)
│   ├─ /schedule → Schedule.jsx
│   ├─ /survey → Survey.jsx
│   ├─ /eventmap → EventMap.jsx
│   ├─ /bingo → BingoGame.jsx (bingoGame.jsx)
│   ├─ /avatar → AvatarCreator.jsx
│   └─ /mappage → MapPage.jsx
├─ /host → Host.jsx
├─ EnterCode.jsx (Standalone)
└─ Home.jsx (Standalone)

```

Styling & Layout

App-Wide Layout (App.css)

- **Header** - Dark green (#053416) navigation bar with HereToo branding
- **Footer** - Matching footer with copyright info
- **Main Content** - Flexible area between header and footer

- **Fullscreen Mode** - EventMap uses fullscreen layout option (no padding)

Component-Specific Styling

- **BingoGame** - Uses `bingoGame.css` for modal and card layouts
 - **Survey** - Custom form styling with validation feedback
 - **EventMap** - Canvas/SVG-based spatial layout
-

Key Technologies

- **React Router v6** - Client-side routing and navigation
 - **Zoom SDK** - Integration for host features and app launch
 - **React State** - Local state management for game logic, modals, scores
 - **CSS Custom Properties** - Color scheme and theme consistency
 - **Responsive Design** - Mobile-friendly layouts with media queries
-

Data Flow with Backend

Participant Journey Data Flow

1. **Enter Code** → Backend lookup via `/meetings/by-code/{user_code}`
2. **Survey Submit** → Backend upsert via `POST /players/{player_id}/survey`
3. **Bingo Interaction** → Local state tracking (currently client-side only)

Backend Integration Points

- `GET /players/expertise` - Fetch available expertise options for survey
 - `POST /players/{player_id}/survey` - Submit survey responses
 - `GET /meetings/by-code/{user_code}` - Validate and retrieve meeting details
 - *(Future)* Match algorithm to populate Bingo game participants
-

Development Notes

State Management

- Currently uses React `useState` for local component state

- Bingo game score and interactions tracked locally

Responsive Design

- Breakpoint at 768px for tablet
 - Breakpoint at 1024px for larger displays
 - Modals have max-height with overflow scrolling for small screens
-

Backend Documentation (FastAPI)

Overview

This backend is built with FastAPI and SQLAlchemy, providing a structured API for hosting, joining, and managing interactive meetings and surveys.

It includes modular routes for:

- **Dev utilities** (database resets)
- **Host operations** (Zoom-linked games)
- **Meetings** (creation and lookup)
- **Players** (joining meetings + surveys)
- **Interest-based matching** (users matched based on interests and area of expertise)

The project uses SQLite (local) or PostgreSQL (planned) and is designed to support interactive “game-like” meeting sessions between hosts and players.

Here is a rough sketch of how the game joining works on the backend:

Interest-based matching algorithm

This algorithm heavily uses Pandas for computationally efficient handling of DataFrames with respect to the expected size of the total number of participants at any one time.

This algorithm takes the Jaccard similarity between the interests of any two users and calculates a similarity score. This similarity score is then multiplied with its respective weight which can be adjusted by us. This algorithm also finds any similar area of expertise shared between said two users and likewise, derives a score. This score, multiplied with its respective weight, is then added to the already weighted interest score.

Pertaining to the area of expertise, specific faculties will be matched to one another if they fall under a similar bigger category. For example, Medicine, Dentistry, and Nursing all falls under Health Sciences. As such, users will be able to be matched to others in the same or similar field.

The final resulting score through addition signifies the amount of similarity between two users based on both their interests and expertise. With this, we can derive the top k recommendations of any users with those recommendations being sorted according to the similarity score.

This algorithm will be integrated into our (later) bingo networking game algorithm whereby users are put into breakout rooms with their top five recommendations. Note that the number of recommendations can be adjusted as wished.

models.py

Defines all SQLAlchemy ORM models and relationships.

Meeting

Represents a scheduled meeting or session.

Field	Type	Description
id	String (UUID)	Primary key
user_code	String	Code for users to join
admin_code	String	Admin code for host controls
title	String	Meeting title
description	Text	Optional description
host	String	Host name or identifier
scheduled_time	DateTime	Meeting start
duration_minutes	Integer	Duration in minutes
physical_location	String	Optional
online_link	String	Optional
created_at / updated_at	DateTime	Timestamps
Relations	players → list of Player	

Game

Represents a hosted Zoom game session.

Field	Type	Description
<code>id</code>	<code>String</code>	Unique numeric game code (e.g., 4 digits)
<code>zoom_host_id</code>	<code>String</code>	Host's Zoom participant ID
<code>zoom_meet_id</code>	<code>String</code>	Zoom meeting ID
<code>status</code>	<code>String</code>	"OPEN", "CLOSED", etc.
<code>expertise</code>	<code>JSON</code>	Selected expertise areas (optional)
<code>time_start</code> / <code>time_end</code>	<code>DateTime</code>	Game start and end times
<code>created_at</code> / <code>updated_at</code>	<code>DateTime</code>	Timestamps

Player

Represents a participant joining a meeting.

Field	Type	Description
<code>id</code>	<code>Integer</code>	Auto-increment PK
<code>zoom_participant_id</code>	<code>String</code>	Player's Zoom participant UUID
<code>name</code>	<code>String</code>	Display name
<code>meeting_id</code>	FK → <code>Meeting.id</code>	Meeting reference
<code>expertise_id</code>	FK → <code>Expertise.id</code>	Optional
<code>score</code>	<code>Integer</code>	Player's score
<code>time_finished</code>	<code>DateTime</code>	Completion timestamp
<code>interests</code>	Many-to-Many	Linked via <code>PlayerInterest</code>

Expertise

Defines available expertise areas (seeded).

Field	Type	Description
-------	------	-------------

Field	Type	Description
<code>id</code>	<code>String</code> (UUID)	Primary key
<code>name</code>	<code>String</code>	Unique expertise name

Interest

Stores normalized interest tags.

Field	Type	Description
<code>id</code>	<code>String</code> (UUID)	Primary key
<code>name</code>	<code>String</code>	Normalized interest name

PlayerInterest

Join table mapping `Player` ↔ `Interest` .

Field	Type	Description
<code>player_id</code>	FK → <code>Player.id</code>	Composite PK
<code>interest_id</code>	FK → <code>Interest.id</code>	Composite PK

`schemas.py`

Defines **Pydantic** schemas for request validation and response serialization.

Game Schemas

- `GameCreate` – For host creating a new game.
- `Game` – For returning game details.

Expertise & Interest Schemas

- `ExpertiseOut` / `InterestOut` – Public response models.

Player Schemas

- **PlayerCreateWithCode** – Joining a meeting using `meeting_code` .
- **PlayerOut** – Returns full player object with related expertise and interests.

Survey Schema

- **SurveyIn** – Player-submitted survey form.
 - `expertise_id` : optional
 - `interests` : list of strings

Meeting Schemas

- **MeetingCreate** – Used by admin or system to create a meeting.
- **MeetingOut** – Returns full meeting with codes and timestamps.

Routes

/dev – Developer Utilities

Endpoint	Method	Description
<code>/dev/all</code>	GET	Returns contents of all database tables (debug).
<code>/dev/drop-all</code>	GET	Drops and recreates all tables. Useful during early testing.

/host – Host Operations

Handles authenticated host actions (Zoom apps).

Endpoint	Method	Description
<code>/host/ping</code>	POST	Basic endpoint for testing user header parsing.
<code>/host/{host_id}/games</code>	GET	Lists all games owned by a specific Zoom host. Requires header validation.
<code>/host/{host_id}/create</code>	POST	Creates a new game record for the host. Generates unique 4-digit ID.
<code>/host/games/all</code>	GET	Returns all games (for development).

Authentication Header

- Each host request expects a `User` header with JSON payload:

```
{  
  "participantUUID": "xxxx",  
  "role": "host"  
}
```

- Role is validated via `get_user()` dependency.

`/meetings` – Meeting Management

Endpoint	Method	Description
<code>/meetings</code>	POST	Creates a new meeting with generated <code>user_code</code> and <code>admin_code</code> .
<code>/meetings/by-code/{user_code}</code>	GET	Fetch meeting details by the user join code.
<code>/meetings/{meeting_id}/players</code>	GET	Lists all players belonging to the given meeting.

Used for the **“Enter Code”** and **Survey pre-join** flow in frontend.

`/players` – Player and Survey Flow

Handles player joining and post-join survey submission.

Endpoint	Method	Description
<code>/players/join/{game_id}</code>	POST	Allows a player to join a game lobby using a join code.
<code>/players/{player_id}/survey</code>	POST	Submits a player’s interests and expertise. Updates relationships.
<code>/players/expertise</code>	GET	Returns a the list of expertise areas (Design, Software, etc.). to be moved to new routes file called expertise.py in future.

FUTURE PLANS: Routes to Add for Full REST Coverage

- Some routes remain unfinished in the interest of speeding up developement. We are currently missing deletes and patches for some models and this will be added in the future as outlined

below.

- We still need to intergrate the backend interest matching components and setup the proper routes for them.
- We have been restructuring our code and thus this still needs intergration

Meetings

Current endpoints	Missing RESTful endpoints to add	Notes
<code>POST /meetings</code> - create a meeting	<code>GET /meetings</code> - list meetings	Useful for admin/host dashboards.
<code>GET /meetings/by-code/{user_code}</code> - fetch by join code	<code>GET /meetings/{id}</code> - fetch by ID	Needed for canonical resource fetch.
<code>GET /meetings/{meeting_id}/players</code> - list players in meeting	<code>PATCH /meetings/{id}</code> - partial update (title, time, links, etc.)	Prefer PATCH for partial updates; support PUT if you want full replace.
	<code>DELETE /meetings/{id}</code> - delete meeting	Should cascade or reject when players exist (you already have <code>cascade="all, delete-orphan"</code> on <code>Meeting.players</code>).
	(optional) <code>POST /meetings/{id}/regenerate-codes</code>	If you need to rotate <code>user_code</code> / <code>admin_code</code> .

Players

Current endpoints	Missing RESTful endpoints to add	Notes
<code>POST /players/join</code> - create player (via meeting <code>user_code</code>)	<code>GET /players</code> - list players	Support filters: <code>?meeting_id=...</code> , <code>?zoom_participant_id=...</code>
<code>POST /players/{player_id}/survey</code> - update expertise/interests	<code>GET /players/{id}</code> - fetch single player	For detail views.
	<code>PATCH /players/{id}</code> - update name, expertise, interests, score	You already have logic to upsert interests; expose it here too.

Current endpoints	Missing RESTful endpoints to add	Notes
	<code>DELETE /players/{id}</code> - remove player	Consider behavior if game/session is running.

Games

Current endpoints	Missing RESTful endpoints to add	Notes
<code>GET /host/{host_id}/games</code> - list host's games	<code>POST /games</code> - create game	Current create is tied to host route <code>POST /host/{host_id}/create</code> . Consider a canonical <code>/games</code> endpoint (auth still checks host).
<code>POST /host/{host_id}/create</code> - create game for host	<code>GET /games</code> - list games	Useful for admin tooling (filter by <code>host_id</code>).
<code>GET /host/games/all</code> - dev list	<code>GET /games/{id}</code> - fetch by ID	Canonical fetch by game ID.
	<code>PATCH /games/{id}</code> - update status (<code>OPEN / CLOSED</code>), <code>time_start</code> , <code>time_end</code> , <code>expertise</code>	Aligns with <code>models.Game</code> fields.
	<code>DELETE /games/{id}</code> - delete game	Restrict to host/owner.

Expertise

Current endpoints	Missing RESTful endpoints to add	Notes
<code>GET /players/expertise</code> - list seeded expertise	<code>POST /expertise</code> - create	For admin to manage taxonomy.
	<code>GET /expertise/{id}</code> - fetch one	

Current endpoints	Missing RESTful endpoints to add	Notes
	PATCH /expertise/{id} - rename	
	DELETE /expertise/{id} - delete	Enforce FK constraints with players.expertise_id.

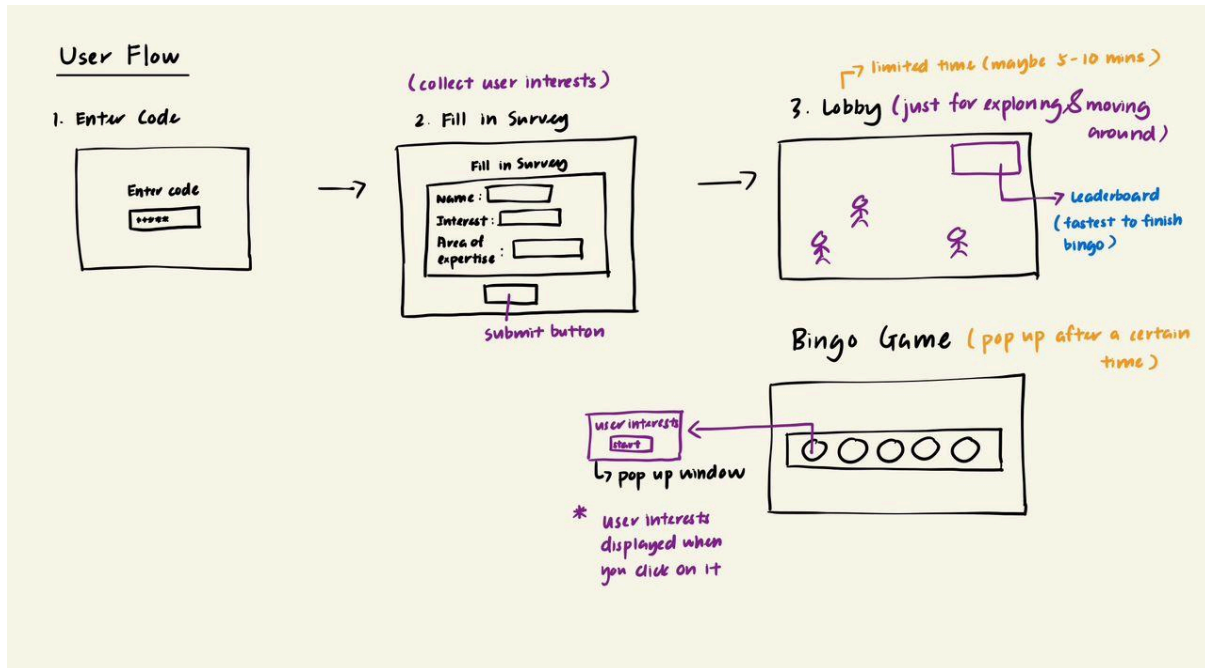
Interests

Current endpoints	Missing RESTful endpoints to add	Notes
(created on-the-fly from survey via helpers)	GET /interests - list all	Visibility into the tag set (you normalize on write).
	POST /interests - create explicit interest	Optional, since you already upsert from survey.
	GET /interests/{id} - fetch one	
	PATCH /interests/{id} - rename (maintain normalized unique name)	Keep normalization rules consistent with _normalize_interest.
	DELETE /interests/{id} - delete	Consider behavior when linked via player_interests M2M.

ngrok http --url=electric-sure-sawfish.ngrok-free.app 5173

AI Usage: AI has been used in this documentation mostly to create the annoying markdown tables which are very tedious to make. ChatGPT-5 and Claude Opus 4.1 were used for this purpose on several instances and to re-write documentation to be more professional.

App flow Diagram



- User shall enter code to enter a certain meeting and will be obliged to fill in a survey
- Survey; this is to obtain their interest. The survey will be static questions (for now) with fields of name, interest (where they can fill in anything, a hobby, a pet, etc) and expertise (a drop down, filled expertise field quite vast within the database)
- After submitting the survey, user will be directed to lobby, where they can see all participants name and interest
- Bingo game will only appear at ice breaking session, enabled by host

Bingo game

- Will be filled with 5 names of people user to interact, later on will be appointed to zoom (if one of them online or both online) or direct meet if offline

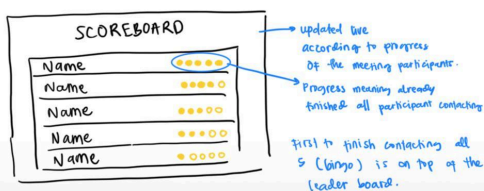
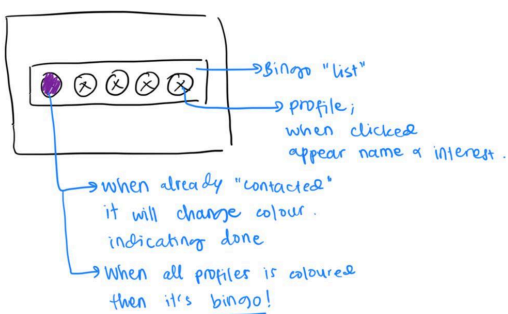
Conversation question will prompt from the interaction screen and both user will need to fill in the answer to end the interact session and obtain score

finish interacting with all 5 will BINGO!

Score will be live updated in the scoreboard for motivation purposes, first one to BINGO!

Will be on top of the scoreboard

Human Bingo



Above: Diagram of Human Bingo

